PhishLens Supervisor Briefing

A simple explanation of what Emma built, how the algorithms work, and how the numbers are produced

---

Student: Emma
Project: PhishLens - Explainable Ensemble Machine Learning for Phishing URL Detection
Purpose: A simple, accurate explanation of what Emma built, which algorithms were used, how they produce numbers, and how this answers supervisor feedback.

This briefing is written in plain language so Emma can explain the technology to anyone, including non-technical readers and examiners.

---

1. What Emma built, in one sentence

Emma built a local website (Streamlit) where a person pastes a URL. Three machine-learning models look at the URL, vote together, say whether it is phishing or legitimate, show how sure they are, and then explain why in everyday English using SHAP.

The system looks at the URL text only. It does not visit the webpage, open email, or scan files.

---

2. The most important idea: 70% is not accuracy

The supervisor asked Emma to understand how the algorithms come up with numbers, and why a result might be 70% instead of 100%.

These are two different numbers.

Number shown	What it means	Simple example
Probability / confidence	How sure the model is about this one URL	"This link looks about 70% phishing."
Accuracy	How often the model was right overall...	"On 104 exam URLs, it got 100% right."

70% probability is not 70% accuracy.

- 70% means: "I am fairly sure, but not certain, about this URL."
- 100% accuracy means: "On the test paper of 104 labelled URLs, I got them all right."

Why not always 100% on a single URL?

Because a real URL can look mixed:

- it uses HTTPS (safer sign)
- but also has words like login or verify (scam sign)
- and a long, messy path (scam sign)

The models add those clues together. If some clues say "safe" and some say "danger," the score may land at 70%, not 99%. That is not a failure. That is the model being honest about mixed evidence.

Sentence Emma can say:

Accuracy is a school-exam score for the whole test set. Probability is how confident the model is about one new URL.

---

3. What Emma did, step by step

1. Collect labelled URLs (phishing vs legitimate).
2. Split them into train / validation / test (60% / 20% / 20%).
3. Turn each URL into 34 numbers (length, hyphens, suspicious words, HTTPS, and so on).
4. Train three models on those numbers.
5. Combine them with soft voting (they average their probabilities).
6. Test them on URLs they never saw during training.
7. Use SHAP to explain which features pushed the answer toward phishing or legitimate.
8. Put it in a Streamlit app so a person can paste a URL and see the result plus a plain-English explanation.

---

4. Base models Emma used

Emma used three base models:

1. Logistic Regression
2. Random Forest
3. Support Vector Machine (SVM)

Then a fourth component on top:

4. Soft-voting ensemble (VotingClassifier) - averages the three models' probabilities.

That is exactly what the code trains.

---

5. How each algorithm works and how it produces a number

Think of the 34 features as 34 clues about a URL.

5.1 Logistic Regression - a weighted checklist

Imagine each clue has a weight.

- Long confusing URL: maybe +0.8 toward phishing
- Uses HTTPS: maybe -0.4 toward legitimate
- Contains login / verify: maybe +1.2 toward phishing

The model adds those weighted clues into one score, then squeezes that score into a number between 0 and 1 using a sigmoid (an S-shaped curve).

- Close to 1 = phishing
- Close to 0 = legitimate
- 0.70 = "70% chance this is phishing"

What the technical sentence means

The sentence:

"Logistic Regression takes the 34 features extracted from the URL, applies learned coefficients to those features, calculates a score and converts that score into a probability using the sigmoid function. The probability is then used to classify the URL as phishing or legitimate."

In everyday words:

It gives each clue a weight, adds them up, then turns the total into a percentage-like probability. If that probability is above 50%, the URL is called phishing. If it is below 50%, it is called legitimate.

5.2 Random Forest - many small decision trees voting

A decision tree is like a flowchart:

- Is the URL very long? Yes/No
- Does it contain suspicious words? Yes/No
- Does it look like a fake brand name? Yes/No

Random Forest builds many trees (Emma used 200). Each tree looks at slightly different clues, then they vote.

- If 160 of 200 trees say phishing, the probability is high.
- If the trees are split, the probability might be around 70%.

Sentence Emma can say:

Random Forest is a crowd of decision trees. The crowd vote becomes the probability.

5.3 Support Vector Machine (SVM) - draw a border between safe and scam

SVM looks at URLs as points in space.

- Phishing URLs on one side
- Legitimate URLs on the other

It draws the widest possible gap between the two groups. Emma used an RBF kernel, which means the border can be curved, not only a straight line.

For a new URL, SVM asks: which side of the border is it on, and how far from the line?

- Far into the phishing side = high phishing probability
- Near the line = more like 60-70% (uncertain)

Sentence Emma can say:

SVM draws a smart border between phishing and legitimate. Distance from that border becomes confidence.

5.4 Ensemble (soft voting) - ask three experts and average them

Each model outputs a phishing probability, for example:

- Logistic Regression: 62%
- Random Forest: 81%
- SVM: 74%

Ensemble is approximately (62 + 81 + 74) / 3 = 72%.

Then:

- If phishing probability is above 50% -> label = Phishing
- If below 50% -> label = Legitimate
- The bigger number is shown as confidence

That is how the app comes up with those numbers.

---

6. Why a URL might show 70% and not 100%

Emma can give these honest reasons:

1. The URL has mixed signals - some features look safe, some look dangerous.
2. The three models may disagree - averaging them can pull a 99% score down toward 70%.
3. 100% would mean "I am completely sure." Real URLs are rarely that clean.
4. Accuracy can still be high even when an individual URL is not 100% confident.

---

7. What Emma's evaluation numbers actually show

On the latest held-out test set (104 URLs):

Model	Accuracy	Simple meaning
Logistic Regression	about 97%	almost always right, missed a few
Random Forest	100%	right on all 104 test URLs
SVM	100%	right on all 104 test URLs
Ensemble	100%	right on all 104 test URLs

Ensemble also recorded:

- Precision: 1.000
- Recall: 1.000
- F1: 1.000
- ROC-AUC: 1.000
- Confusion matrix: TN=44, FP=0, FN=0, TP=60

Important honesty for the supervisor:

Emma should not say "the ensemble beat everyone on every metric."

Say this instead:

On this small test set, Random Forest, SVM and the ensemble all scored 100%. Logistic Regression scored about 97%. The ensemble matched the strongest models and beat Logistic Regression. Because the dataset is small, 100% must be treated carefully. It is good for the prototype, not proof the system is perfect in the real world.

---

8. Is the technical scope in line with what Emma used?

Yes.

Supervisor list	Emma's project
Python	Yes
Scikit-learn	Yes (Logistic Regression, Random Fore...
Pandas	Yes (data tables)
NumPy	Yes (numbers / arrays)
SHAP	Yes (explanations)

Streamlit	Yes (the website)
Google Colab / Jupyter	Yes - notebooks/phishlens_colab.ipynb
VS Code / local Python	Yes - local prototype

Also used around that stack: Matplotlib/Seaborn for plots, and Joblib to save trained models.

---

9. Confirm the evaluation statements

Yes. That is Emma's flow.

1. The three individual classifiers are first evaluated independently.
   Logistic Regression, Random Forest and SVM are each trained and scored on the test set.

2. Their results are then compared with the ensemble voting classifier.
   This is stored in evaluation/metrics.json and shown on the Model Performance page.

3. The evaluation therefore checks whether the ensemble provides measurable improvement over the individual models.
   That is the purpose. On this run, the ensemble matched the best models (Random Forest and SVM) and beat Logistic Regression. Emma should say this honestly.

4. SHAP outputs are examined to determine whether the system can provide meaningful information about the features influencing individual predictions.
   Yes. The app shows SHAP charts, ranked features, and a "Why we said Phishing / Legitimate" plain-English explanation.

---

10. Confirm the artefact components

Yes. Emma's artefact consists of these main components:

1. URL input interface
2. URL feature extraction
3. Data preprocessing pipeline
4. Logistic Regression model
5. Random Forest model
6. Support Vector Machine model
7. Ensemble voting classifier
8. Prediction component
9. SHAP explainability component
10. Streamlit user interface

---

11. How a prediction is made in the app

When Emma (or a user) pastes a URL:

1. The URL is checked and cleaned.
2. 34 features are extracted from the URL text.
3. Each of the three models produces a phishing probability.

4. Soft voting averages those three probabilities.
5. If the average is above 50%, the verdict is Phishing. Otherwise it is Legitimate.
6. The app also shows risk level:
   - Low: 0% to 30%
   - Medium: 30% to 60%
   - High: 60% to 85%
   - Critical: 85% to 100%
7. SHAP explains which clues pushed the score up or down.
8. The app turns those clues into a human-readable explanation and advice.

---

12. What makes PhishLens different, in simple words

Most other sites say: "Safe" or "Dangerous."

Emma's site also says phishing or legitimate, but then it explains the decision in everyday language, for example:

- this link looks risky because the address looks long or confusing
- it uses words like login or verify that scammers often use
- here is what you should do next

One-line difference:

Other tools often give a yes/no answer. PhishLens gives a yes/no answer plus a clear explanation of why.

It is a research prototype for transparency and understanding, not a commercial replacement for Google Safe Browsing or VirusTotal.

---

13. 30-second speech Emma can use with the supervisor

I built PhishLens, a local Streamlit app that classifies a URL as phishing or legitimate.

I extract 34 numbers from the URL text only.

I train three models: Logistic Regression, Random Forest and SVM.

Logistic Regression weights the clues and turns the total into a probability.

Random Forest is many decision trees voting.

SVM draws a boundary between safe and scam URLs.

I combine them with soft voting by averaging their probabilities.

If the average phishing score is above 50%, I call it phishing.

A 70% score means the model is 70% sure about that one URL. That is not the same as 70% accuracy. Accuracy is how many test URLs it got right overall.

SHAP then shows which features pushed the decision, in plain English.

---

14. Short glossary

Term	Simple meaning
Feature	A clue extracted from the URL, such a...
Probability	How sure the model is about one URL
Accuracy	How often the model was right on many...
Ensemble	Several models combined; here they av...
Soft voting	Averaging the three models' probabili...
SHAP	A method that shows which clues pushe...
Sigmoid	An S-shaped curve that turns a score ...
XAI	Explainable Artificial Intelligence -...

End of briefing - written for Emma to explain PhishLens in simple, accurate language.